

Guide de Déploiement VPS

Multi-applications · HTTPS automatique · Docker

appedu.tech

Hébergeur	Hostinger KVM1
Système	Ubuntu 24.04 LTS
Reverse proxy	Caddy 2 (HTTPS automatique)
Conteneurs	Docker + Docker Compose
Base de données	PostgreSQL 16
Domaine	appedu.tech

Table des matières

- I** Préparation du VPS (une seule fois)
- II** Caddy partagé (une seule fois)
- III** DNS — Sous-domaines
- IV** Déployer une application
- V** Ajouter une nouvelle application
- VI** Mettre à jour une application
- VII** Référence rapide

I Préparation du VPS (une seule fois)

Ces étapes ne sont réalisées qu'une seule fois, à la création du serveur. Elles préparent l'environnement pour accueillir toutes vos applications.

1 Créer le VPS sur Hostinger

Dans hPanel → **VPS** → votre serveur :

Option	Valeur recommandée
Plan	KVM1 (1 vCPU · 4 Go RAM · 50 Go NVMe)
Système d'exploitation	Ubuntu 24.04 LTS
Clé SSH	Ajouter votre clé publique (~/.ssh/id_ed25519.pub)
Domaine gratuit	Réclamer appedu.tech via hPanel → Domaines

■ Notez l'adresse IP publique affichée dans le dashboard — vous en aurez besoin pour le DNS.

2 Connexion et sécurisation initiale

Connectez-vous en SSH et créez un utilisateur dédié :

```
# Connexion initiale en root
ssh root@<IP_DU_VPS>

# Mise à jour du système
apt update && apt upgrade -y

# Créer un utilisateur dédié
adduser deploy
usermod -aG sudo deploy
rsync --archive --chown=deploy:deploy ~/.ssh /home/deploy

# Activer le firewall
ufw allow OpenSSH
ufw allow 80/tcp
ufw allow 443/tcp
ufw enable

# Déconnexion
exit
```

■ Dans hPanel → Firewall, vérifiez que les ports 22, 80 et 443 sont également ouverts.

3 Installer Docker

Reconnectez-vous avec l'utilisateur *deploy* et installez Docker :

```
ssh deploy@<IP_DU_VPS>

# Installation officielle Docker
curl -fsSL https://get.docker.com | sh

# Ajouter l'utilisateur au groupe docker
sudo usermod -aG docker $USER

# Reconnexion pour prise en compte
exit
ssh deploy@<IP_DU_VPS>

# Vérification
docker --version && docker compose version
```

4 Créer le réseau Docker partagé

Ce réseau permet à Caddy de communiquer avec le frontend de chaque application. Il est créé une seule fois et persiste même si les conteneurs redémarrent.

```
docker network create caddy-net
```

II Caddy partagé (une seule fois)

Caddy est le reverse proxy qui gère le HTTPS automatique (Let's Encrypt) et route les requêtes vers chaque application selon le sous-domaine. Un seul Caddy tourne sur le VPS et dessert toutes les applications.

1 Créer la structure de dossiers

```
mkdir ~/caddy && cd ~/caddy
```

2 Créer ~/caddy/Caddyfile

Ce fichier définit le routage. Chaque nouvelle application ajoute une entrée. Le nom de conteneur suit le pattern **<dossier>-<service>-1**.

```
# Exemple avec deux applications
exopy.appedu.tech {
    reverse_proxy exopy_multi_admin-frontend-1:80
}

autreapp.appedu.tech {
    reverse_proxy autreapp-frontend-1:80
}
```

■ Au démarrage, ne mettez que les applications déjà déployées. Ajoutez les autres au fur et à mesure.

3 Créer ~/caddy/docker-compose.yml

```

services:
  caddy:
    image: caddy:2-alpine
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
      - "443:443/udp"          # HTTP/3
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
      - caddy_config:/config
    networks:
      - caddy-net

volumes:
  caddy_data:
  caddy_config:

networks:
  caddy-net:
    external: true          # réseau créé à l'étape précédente

```

4 Lancer Caddy

```

cd ~/caddy
docker compose up -d

# Vérification
docker ps | grep caddy

```

III DNS — Sous-domaines

Chaque application obtient un sous-domaine d'appedu.tech. La configuration se fait dans hPanel → Domaines → appedu.tech → DNS Zone.

Pour chaque application, ajoutez un enregistrement A :

Champ	Valeur	Exemple (exopy)
Type	A	A
Nom	sous-domaine de l'app	exopy
IP	IP publique du VPS	<IP_DU_VPS>
TTL	Auto (ou 3600)	3600

Vérifier la propagation DNS :

```

dig exopy.appedu.tech +short
# Doit retourner l'IP du VPS

```

■ ■ Le DNS doit être propagé AVANT de lancer Caddy avec un nouveau sous-domaine, sinon Let's Encrypt ne pourra pas émettre le certificat.

IV Déployer une application

Exemple complet avec **exopy_multi_admin** sur **exopy.appedu.tech**. La même procédure s'applique à toute nouvelle application.

1 Transférer le projet (depuis votre machine locale)

Exécutez cette commande dans un terminal sur votre ordinateur :

```
rsync -avz \
  --exclude '.git' \
  --exclude 'node_modules' \
  --exclude '.env' \
  /chemin/local/exopy_multi_admin/ \
  deploy@<IP_DU_VPS>:~/exopy_multi_admin/
```

■ Cette commande s'exécute sur votre Mac, pas sur le serveur.

2 Configurer le fichier .env

```
ssh deploy@<IP_DU_VPS>
cd ~/exopy_multi_admin
cp .env.example .env
nano .env
```

Contenu du .env pour la production :

```
# Port frontend (exposé uniquement en local, Caddy s'en charge)
FRONTEND_PORT=127.0.0.1:8080

# CORS : votre sous-domaine
CORS_ORIGINS=https://exopy.appedu.tech

# Base de données
POSTGRES_PASSWORD=<mot_de_passe_fort>

# JWT — générez avec : openssl rand -hex 32
SECRET_KEY=<valeur_aléatoire>

# Super-administrateur
ADMIN_USERNAME=admin
ADMIN_PASSWORD=<mot_de_passe_fort>

# OpenRouter (IA)
OPENROUTER_API_KEY=sk-or-...
```

■ ■ **FRONTEND_PORT** doit être 127.0.0.1:8080 (sans :80 final — docker-compose.yml l'ajoute automatiquement).

3 Créer docker-compose.override.yml

Ce fichier connecte le frontend au réseau Caddy et remplace le `docker-compose.prod.yml` (qui ne doit plus être utilisé en mode multi-app).

```
# ~/exopy_multi_admin/docker-compose.override.yml
services:
  frontend:
    networks:
      - app
      - caddy-net          # visible par Caddy

networks:
  caddy-net:
    external: true
```

■ Docker Compose fusionne automatiquement `docker-compose.yml` et `docker-compose.override.yml`. Pas besoin de les spécifier manuellement.

4 Ajouter l'application dans le Caddyfile

Sur le serveur, éditez `~/caddy/Caddyfile` :

```
nano ~/caddy/Caddyfile

# Ajoutez :
exopy.appedu.tech {
    reverse_proxy exopy_multi_admin-frontend-1:80
}
```

```
# Trouver le nom exact du conteneur si nécessaire :
docker ps --format "table {{.Names}}\t{{.Image}}"
```

5 Lancer l'application

```
cd ~/exopy_multi_admin
docker compose up -d --build
```

Recharger Caddy sans coupure :

```
docker exec caddy caddy reload --config /etc/caddy/Caddyfile
```

6 Vérifications

```
# Tous les conteneurs sont en marche ?
docker ps

# HTTPS fonctionnel ?
curl -I https://exopy.appedu.tech

# Logs en temps réel
docker compose logs -f
```


V Ajouter une nouvelle application

Pour chaque nouvelle application, répétez ces 4 étapes dans l'ordre :

① DNS

Dans hPanel → DNS Zone → ajouter un enregistrement A :

```
Type : A | Nom : autreapp | IP : <IP_DU_VPS> | TTL : 3600
```

② Transférer et configurer l'application

Même procédure que la Section IV (rsync, .env, docker-compose.override.yml).

③ Mettre à jour le Caddyfile

Ajoutez une entrée dans ~/caddy/Caddyfile :

```
autreapp.appedu.tech {
    reverse_proxy autreapp-frontend-1:80
}
```

④ Recharger Caddy

Après avoir lancé l'application (docker compose up -d --build) :

```
docker exec caddy caddy reload --config /etc/caddy/Caddyfile
```

■ Structure finale sur le serveur :

```
~/caddy/ → Caddy partagé (docker-compose.yml + Caddyfile)
~/exopy_multi_admin/ → App 1 (docker-compose.yml + override + .env)
~/autreapp/ → App 2 (docker-compose.yml + override + .env)
```

VI Mettre à jour une application

Pour déployer une nouvelle version de votre code sans reconfigurer le serveur :

1 Transférer les nouveaux fichiers (depuis votre Mac)

```
rsync -avz \
  --exclude '.git' \
  --exclude 'node_modules' \
  --exclude '.env' \
  /chemin/local/exopy_multi_admin/ \
  deploy@<IP_DU_VPS>:~/exopy_multi_admin/
```

2 Rebuild et redémarrer les conteneurs

```
ssh deploy@<IP_DU_VPS>
cd ~/exopy_multi_admin

# Rebuild et redémarrage
docker compose up -d --build

# Nettoyage des anciennes images
docker image prune -f
```

■ Caddy continue de fonctionner pendant le rebuild — le temps d'indisponibilité est minimal.

3 Vérifier le bon déroulement

```
docker ps
docker compose logs --tail=50
```

Sauvegarde de la base de données (avant une mise à jour importante) :

```
# Créer un dump PostgreSQL
docker exec $(docker ps -qf "name=postgres") \
  pg_dump -U exopy exopy > backup_$(date +%Y%m%d_%H%M).sql

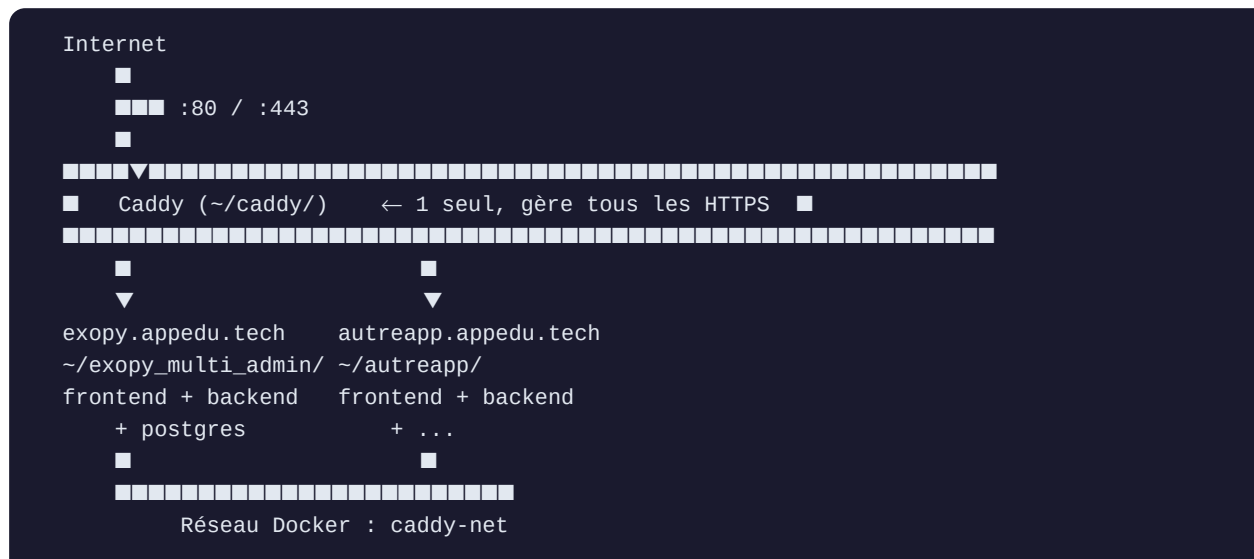
# Restaurer si nécessaire
cat backup_20260524_1200.sql | \
  docker exec -i $(docker ps -qf "name=postgres") psql -U exopy exopy
```

VI I Référence rapide

Commandes essentielles

Action	Commande
Voir tous les conteneurs	<code>docker ps</code>
Logs d'une app	<code>docker compose logs -f</code>
Recharger Caddy	<code>docker exec caddy caddy reload --config /etc/caddy/Caddyfile</code>
Redémarrer une app	<code>docker compose restart</code>
Rebuild complet	<code>docker compose up -d --build</code>
Nettoyer images inutiles	<code>docker image prune -f</code>
Noms des conteneurs	<code>docker ps --format "table {{.Names}}\t{{.Image}}"</code>
Backup PostgreSQL	<code>docker exec \$(docker ps -qf "name=postgres") pg_dump -U exopy exopy > backup.sql</code>

Architecture réseau



Checklist — nouvelle application

- Enregistrement DNS A créé dans hPanel (sous-domaine → IP du VPS)
- Propagation DNS vérifiée (dig .appedu.tech +short)
- Fichiers transférés via rsync (sans .env)
- Fichier .env configuré sur le serveur
- docker-compose.override.yml créé (réseau caddy-net)
- Entrée ajoutée dans ~/.caddy/Caddyfile
- docker compose up -d --build lancé
- Caddy rechargé (caddy reload)
- HTTPS vérifié (curl -I https://.appedu.tech)